arXiv:1712.09913v3 [cs.LG] 7 Nov 2018

Visualizing the Loss Landscape of Neural Nets

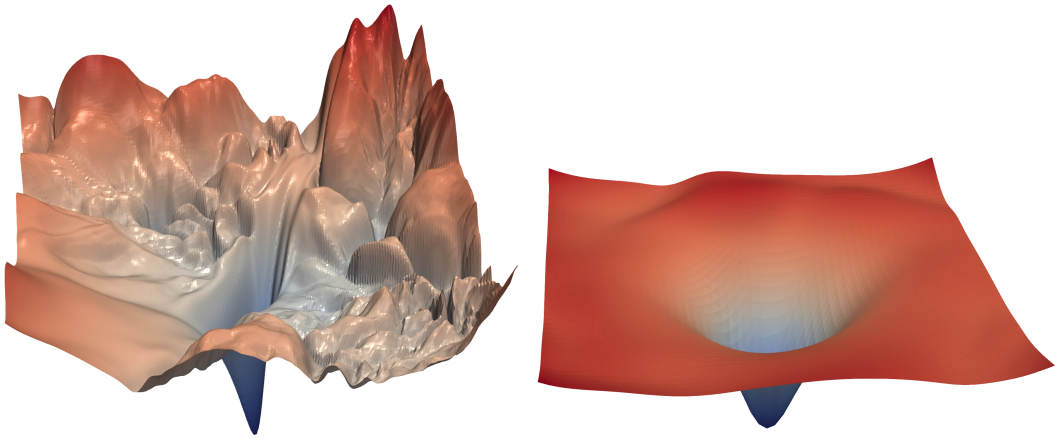
Hao Li[1], **Zheng Xu**[1], **Gavin Taylor**[2], **Christoph Studer**[3], **Tom Goldstein**[1]
[1]University of Maryland, College Park [2]United States Naval Academy [3]Cornell University
{haoli,xuzh,tomg}@cs.umd.edu, taylor@usna.edu, studer@cornell.edu

Abstract

Neural network training relies on our ability to find "good" minimizers of highly non-convex loss functions. It is well-known that certain network architecture designs (e.g., skip connections) produce loss functions that train easier, and well-chosen training parameters (batch size, learning rate, optimizer) produce minimizers that generalize better. However, the reasons for these differences, and their effects on the underlying loss landscape, are not well understood. In this paper, we explore the structure of neural loss functions, and the effect of loss landscapes on generalization, using a range of visualization methods. First, we introduce a simple "filter normalization" method that helps us visualize loss function curvature and make meaningful side-by-side comparisons between loss functions. Then, using a variety of visualizations, we explore how network architecture affects the loss landscape, and how training parameters affect the shape of minimizers.

1 Introduction

Training neural networks requires minimizing a high-dimensional non-convex loss function – a task that is hard in theory, but sometimes easy in practice. Despite the NP-hardness of training general neural loss functions [2], simple gradient methods often find global minimizers (parameter configurations with zero or near-zero training loss), even when data and labels are randomized before training [42]. However, this good behavior is not universal; the trainability of neural nets is highly dependent on network architecture design choices, the choice of optimizer, variable initialization, and a variety of other considerations. Unfortunately, the effect of each of these choices on the structure of the underlying loss surface is unclear. Because of the prohibitive cost of loss function evaluations (which requires looping over all the data points in the training set), studies in this field have remained predominantly theoretical.



(a) without skip connections (b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

32nd Conference on Neural Information Processing Systems (NIPS 2018), Montréal, Canada.

Visualizations have the potential to help us answer several important questions about why neural networks work. In particular, why are we able to minimize highly non-convex neural loss functions? And why do the resulting minima generalize? To clarify these questions, we use high-resolution visualizations to provide an empirical characterization of neural loss functions, and explore how different network architecture choices affect the loss landscape. Furthermore, we explore how the non-convex structure of neural loss functions relates to their trainability, and how the geometry of neural minimizers (i.e., their sharpness/flatness, and their surrounding landscape), affects their generalization properties.

To do this in a meaningful way, we propose a simple "filter normalization" scheme that enables us to do side-by-side comparisons of different minima found during training. We then use visualizations to explore sharpness/flatness of minimizers found by different methods, as well as the effect of network architecture choices (use of skip connections, number of filters, network depth) on the loss landscape. Our goal is to understand how loss function geometry affects generalization in neural nets.

1.1 Contributions

We study methods for producing meaningful loss function visualizations. Then, using these visualization methods, we explore how loss landscape geometry affects generalization error and trainability. More specifically, we address the following issues:

- We reveal faults in a number of visualization methods for loss functions, and show that simple visualization strategies fail to accurately capture the local geometry (sharpness or flatness) of loss function minimizers.
- We present a simple visualization method based on "filter normalization." The sharpness of minimizers correlates well with generalization error when this normalization is used, even when making comparisons across disparate network architectures and training methods. This enables side-by-side comparisons of different minimizers[1].
- We observe that, when networks become sufficiently deep, neural loss landscapes quickly transition from being nearly convex to being highly chaotic. This transition from convex to chaotic behavior coincides with a dramatic drop in generalization error, and ultimately to a lack of trainability.
- We observe that skip connections promote flat minimizers and prevent the transition to chaotic behavior, which helps explain why skip connections are necessary for training extremely deep networks.
- We quantitatively measure non-convexity by calculating the smallest (most negative) eigenvalues of the Hessian around local minima, and visualizing the results as a heat map.
- We study the visualization of SGD optimization trajectories. We explain the difficulties that arise when visualizing these trajectories, and show that optimization trajectories lie in an extremely low dimensional space. This low dimensionality can be explained by the presence of large, nearly convex regions in the loss landscape, such as those observed in our 2-dimensional visualizations.

2 Theoretical Background

Numerous theoretical studies have been done on our ability to optimize neural loss function [5, 4]. Theoretical results usually make restrictive assumptions about the sample distributions, non-linearity of the architecture, or loss functions [16, 31, 40, 36, 9, 39]. For restricted network classes, such as those with a single hidden layer, globally optimal or near-optimal solutions can be found by common optimization methods [35, 26, 38]. For networks with specific structures, there likely exists a monotonically decreasing path from an initialization to a global minimum [32, 15]. Swirszcz et al. [37] show counterexamples that achieve "bad" local minima for toy problems.

Several works have addressed the relationship between sharpness/flatness of local minima and their generalization ability. Hochreiter and Schmidhuber [18] defined "flatness" as the size of the connected region around the minimum where the training loss remains low. Keskar et al. [24] characterize

[1] Code and plots are available at `https://github.com/tomgoldstein/loss-landscape`

flatness using eigenvalues of the Hessian, and propose ϵ-sharpness as an approximation, which looks at the maximum loss in a neighborhood of a minimum. Dinh et al. [7], Neyshabur et al. [30] show that these quantitative measure of sharpness are not invariant to symmetries in the network, and are thus not sufficient to determine generalization ability. Chaudhari et al. [3] used local entropy as a measure of sharpness, which is invariant to the simple transformation in [7], but difficult to accurately compute. Dziugaite and Roy [8] connect sharpness to PAC-Bayes bounds for generalization.

3 The Basics of Loss Function Visualization

Neural networks are trained on a corpus of feature vectors (e.g., images) $\{x_i\}$ and accompanying labels $\{y_i\}$ by minimizing a loss of the form $L(\theta) = \frac{1}{m}\sum_{i=1}^{m} \ell(x_i, y_i; \theta)$, where θ denotes the parameters (weights) of the neural network, the function $\ell(x_i, y_i; \theta)$ measures how well the neural network with parameters θ predicts the label of a data sample, and m is the number of data samples. Neural nets contain many parameters, and so their loss functions live in a very high-dimensional space. Unfortunately, visualizations are only possible using low-dimensional 1D (line) or 2D (surface) plots. Several methods exist for closing this dimensionality gap.

1-Dimensional Linear Interpolation One simple and lightweight way to plot loss functions is to choose two sets of parameters θ and θ', and plot the values of the loss function along the line connecting these two points. We can parameterize this line by choosing a scalar parameter α, and defining the weighted average $\theta(\alpha) = (1-\alpha)\theta + \alpha\theta'$. Finally, we plot the function $f(\alpha) = L(\theta(\alpha))$. This strategy was taken by Goodfellow et al. [13], who studied the loss surface along the line between a random initial guess, and a nearby minimizer obtained by stochastic gradient descent. This method has been widely used to study the "sharpness" and "flatness" of different minima, and the dependence of sharpness on batch-size [24, 7]. Smith and Topin [34] use the same technique to show different minima and the "peaks" between them, while Im et al. [21] plot the line between minima obtained via different optimizers.

The 1D linear interpolation method suffers from several weaknesses. First, it is difficult to visualize non-convexities using 1D plots. Indeed, Goodfellow et al. [13] found that loss functions appear to lack local minima along the minimization trajectory. We will see later, using 2D methods, that some loss functions have extreme non-convexities, and that these non-convexities correlate with the difference in generalization between different network architectures. Second, this method does not consider batch normalization [22] or invariance symmetries in the network. For this reason, the visual sharpness comparisons produced by 1D interpolation plots may be misleading; this issue will be explored in depth in Section 5.

Contour Plots & Random Directions To use this approach, one chooses a center point θ^* in the graph, and chooses two direction vectors, δ and η. One then plots a function of the form $f(\alpha) = L(\theta^* + \alpha\delta)$ in the 1D (line) case, or

$$f(\alpha, \beta) = L(\theta^* + \alpha\delta + \beta\eta) \tag{1}$$

in the 2D (surface) case[2]. This approach was used in [13] to explore the trajectories of different minimization methods. It was also used in [21] to show that different optimization algorithms find different local minima within the 2D projected space. Because of the computational burden of 2D plotting, these methods generally result in low-resolution plots of small regions that have not captured the complex non-convexity of loss surfaces. Below, we use high-resolution visualizations over large slices of weight space to visualize how network design affects non-convex structure.

4 Proposed Visualization: Filter-Wise Normalization

This study relies heavily on plots of the form (1) produced using random direction vectors, δ and η, each sampled from a random Gaussian distribution with appropriate scaling (described below). While the "random directions" approach to plotting is simple, it fails to capture the intrinsic geometry of loss surfaces, and cannot be used to compare the geometry of two different minimizers or two different

[2]When making 2D plots in this paper, batch normalization parameters are held constant, i.e., random directions are not applied to batch normalization parameters.